I built a preprocessing pipeline that brings together several well-known mathematics benchmark datasets into a single, consistent JSONL format for evaluation. The pipeline aggregates the GSM8K test set, the full Hendrycks MATH test set, the MATH-500 subset, and the AIME 2024 problems. Where possible, answers are normalized to a common representation, and each run outputs timestamped files under the /trained dataset directory to avoid overwriting previous results. Due to the anti-bot protections enforced by Art of Problem Solving (AoPS), the AIME problems are supplied manually through a local CSV file (aime2024_input.csv). If this CSV is not available, the pipeline falls back to an official answer table embedded in the script so that processing can still proceed without attempting to scrape protected pages. The project provides both a command-line script (preprocess.py) and an interactive Jupyter notebook (preprocess.ipynb), which implement the same logic and allow users to run the pipeline either end-to-end or step by step for closer inspection.

The preprocessing code is structured around small, clearly documented helper functions. These include norm_number, which standardizes numeric strings by stripping whitespace and removing thousands separators; extract_after_hashes, which retrieves final numeric answers from GSM8K solutions that use the "####" delimiter; and extract_boxed, which extracts LaTeX boxed answers and otherwise searches for a clearly marked answer line in the solution text. Together, these utilities make the pipeline resilient to common formatting differences across datasets, such as extra spaces, commas in numbers, or LaTeX-style answer notation.

Output handling is designed to be safe and repeatable. The write_jsonl function always writes to a timestamped JSONL file so that repeated runs do not clobber existing outputs. The get_math_tags helper assigns consistent subject and difficulty tags to MATH problems, while install_dependencies attempts to automatically install any missing Python dependencies (such as datasets or accelerate). This simplifies setup for first-time users, although running inside a virtual environment is recommended to avoid modifying the global Python installation.

Each dataset is processed through its own dedicated function. The process_gsm8k function loads the openai/gsm8k test split from Hugging Face, extracts final answers using extract_after_hashes, and outputs standardized records containing fields such as problem_id, normalized answer, reference URL, and metadata describing the answer format. The process_math function iterates through the subject splits of the Hendrycks MATH dataset, recovers final answers from solution text, and applies subject- and level-based tags. The process_math500 function performs equivalent normalization for the official MATH-500 test split.

The process_aime2024 function is specifically designed to accommodate AoPS access restrictions. It first checks for the presence of aime2024_input.csv and reads problems (and optional answers) from this file if available. The CSV parsing logic is intentionally defensive: it prefers header-based parsing, but if rows have inconsistent lengths, it reconstructs fields

by treating the first column as the index, the last columns as answer, has_diagrams, and URL, and joining any intermediate columns into the problem statement. When an answer cell is empty or contains non-numeric text, the function falls back to the built-in official answer list to ensure completeness. Each AIME entry stores its normalized answer as a three-digit string (000–999) and includes metadata describing the fixed-format answer type, with a default AoPS reference URL when none is supplied.

To support manual data entry, the helper function create_aime_template generates a starter CSV file (aime2024_input.csv) with placeholder problem text and empty answer fields. Overall execution is exposed in two parallel forms: the main() entry point in preprocess.py, which installs dependencies, prepares the output directory, runs all dataset processors, and prints a summary; and the preprocess.ipynb notebook, which mirrors the same steps for interactive experimentation.

Overall, the project provides a robust and repeatable preprocessing workflow that aligns multiple public mathematics datasets under a unified JSONL schema. It combines defensive parsing and normalization to handle messy source formats, preserves reproducibility through timestamped outputs, and respects anti-scraping constraints by relying on a manual CSV import for AIME 2024 rather than automated data collection.